

Mock Design: A Notification System

Phase 4 · Interview-Ready · Estimated time: 1.5 hours

1. Concept From First Principles: Applying the Framework

Several earlier chapters in this Phase referenced a notification service as a downstream dependency (Chapter 7's message delivery, Chapter 13's order-status updates) without designing it in full. This chapter builds that system properly: a shared, multi-tenant service that other internal systems call to deliver a message to a user across push notifications, SMS, and email — each with genuinely different delivery guarantees, cost, and failure modes.

Step 1 — Clarify requirements

- Functional: accept a notification request from an internal caller (specifying user, message, and channel(s)); deliver via push notification, SMS, and/or email; respect user notification preferences and opt-outs.
- Non-functional: high fan-out volume (a single event, like a flash-sale start, can trigger millions of notifications near-simultaneously); avoid duplicate sends; some channels (SMS, push) cost real money per send, making cost-awareness a genuine non-functional concern, not just an implementation detail.

Step 2 — Estimate scale

Assume a platform-wide notification volume of 200 million notifications/day in steady state (~2,300/second average), but with occasional massive fan-out bursts — a single marketing campaign or order-status batch update triggering tens of millions of notifications within a few minutes. This burst-vs-steady-state gap, similar in shape to Chapter 10's flash-sale framing, is the key scale consideration driving the queue-centric design below.

Step 3 — API

POST /notify {userId, template, data, channels: [PUSH, SMS, EMAIL], idempotencyKey}. Using a **template** (rather than a fully pre-rendered message) lets the notification service handle localization and formatting per channel centrally, and the idempotency key (Phase 2, Ch 9) prevents a calling service's retry from causing a duplicate send to the end user.

Step 4 — High-level design

Calling services → Notification API → a **Message Queue** (Phase 2, Ch 12) — the central architectural decision in this design — which decouples the caller (which just needs to fire-and-forget the request) from the actual delivery work, and naturally absorbs burst traffic by letting consumers drain the queue at a sustainable rate rather than requiring instant synchronous delivery. Separate consumer worker pools, one per channel (Push Worker, SMS Worker, Email Worker), pull from the queue and call the relevant third-party delivery provider (APNs/FCM for push, a telecom SMS gateway, an email delivery provider).

Step 5 — Deep dive: fan-out, retries, and deduplication

Fan-out for bulk sends: a single "notify all users in segment X" request shouldn't be handled as one giant synchronous loop. It's expanded into individual per-user messages published onto the queue — conceptually identical to Chapter 8's fan-out-on-write, just applied to notifications instead of feed posts — letting the many independent worker consumers process the expanded fan-out in parallel, at whatever rate the downstream third-party providers (which often have their own rate limits, Chapter 4) can sustain.

Retry logic with backoff: third-party delivery providers fail transiently and often — a push token might be temporarily unreachable, an SMS gateway might rate-limit you. Failed sends go through retry with exponential backoff and jitter (Phase 3, Ch 14's resilience patterns, applied here directly), and after a bounded number of retries, a failed notification is moved to a dead-letter queue (Phase 2, Ch 12) for investigation rather than retried forever.

Deduplication operates at two levels worth distinguishing explicitly: the idempotency key prevents a calling service's own retry from generating a second notification request, while a separate, short-lived dedup cache (keyed on user+template+time-window) can catch the case where two different internal systems accidentally trigger a very similar notification to the same user within a short window — a real, if less obvious, source of duplicate/spammy notifications in production multi-service systems, worth naming to distinguish this from the simpler single-caller-retry case.

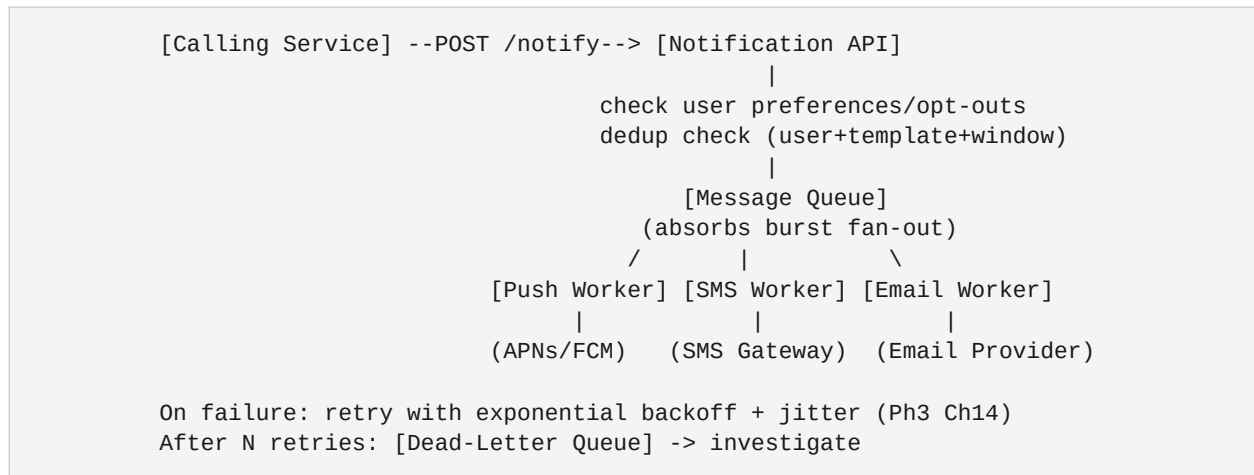
Step 6 — Trade-offs and wrap-up

Volunteer: user notification preferences (opt-outs per channel, quiet hours) must be checked before queuing, not after, to avoid wasted delivery cost and a poor user experience — and note that SMS being the most expensive channel per-send often makes it the fallback (used only if push notification delivery fails or the user has no registered device) rather than a primary channel, a cost-driven design decision worth naming unprompted.

2. Real-World Example / Case Study

Swiggy's and Flipkart's order-status and marketing notification systems, and Razorpay's/PhonePe's transaction-alert SMS and push notifications, are all real production instances of this exact shared, multi-channel, queue-centric architecture — a shared internal notification platform used by many other internal teams' services is extremely common at this scale, precisely because building channel-specific delivery logic (and its retry/dedup/preference handling) separately in every team's service would be significant duplicated effort.

3. Diagram



4. Trade-offs Table

Decision	Alternative	Why this choice
Queue-decoupled, per-channel workers	Synchronous delivery in the API request path	Absorbs burst fan-out; caller doesn't block on slow third-party providers
Fan-out bulk sends onto the queue individually	Loop and send synchronously per user	Enables parallel, independently-rate-limited processing across workers
Exponential backoff + dead-letter queue	Retry forever or fail silently	Bounded retries with investigation path, matches Phase 3 Ch 14's resilience patterns
SMS as a cost-driven fallback, not primary	SMS as a default channel for every notification	SMS is the most expensive channel per-send; push-first reduces cost significantly

5. Common Interview Questions

- Q1: "How does this system absorb a sudden burst of millions of notifications from one campaign?"**
- Model approach: Name the message queue's decoupling role explicitly — the API accepts and expands the bulk request quickly, while independent worker pools drain the queue at whatever sustainable rate the downstream providers allow, rather than requiring instant synchronous delivery of the whole burst.
- Q2: "How do you prevent a user from getting the same notification twice?"**
- Model approach: Distinguish the two deduplication levels — idempotency keys for a calling service's own retries, and a separate short-lived dedup cache for accidental near-duplicate triggers from different internal systems.
- Q3: "Why might you prefer push notifications over SMS as the default channel?"**

Model approach: Name the cost difference directly — SMS carries a real per-send telecom cost while push notifications are effectively free at scale, making push-first with SMS as a fallback a cost-conscious, still user-experience-respecting default.

6. Practice Exercises

- 1 Run the full six-step framework for "design a notification system," out loud, with a timer.
- 2 Design the dedup cache: what key would you use, what TTL, and what edge case does it specifically catch that the idempotency key doesn't?
- 3 Design the retry/dead-letter flow for a push notification that fails 5 consecutive times: what happens at each attempt, and what does the on-call engineer see in the dead-letter queue?

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"System Design Interview - Design a Notification System"	Directly covers multi-channel fan-out and queue-based delivery for this exact problem.
English	"Design a Notification Service - System Design Interview Prep"	Reinforces retry, dead-letter, and preference-handling design concerns.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Bilingual course covering message queues and notification fan-out as part of the broader curriculum.
Hindi	"Complete System Design Roadmap 2026 HLD & LLD" — CodeHelp	Hindi roadmap reinforcing message-queue fundamentals this chapter builds on.

8. Key Takeaways

- A shared, multi-channel notification service is a common piece of internal platform infrastructure, called by many other services.
- Queue-decoupling the API from actual delivery work is what allows the system to absorb massive fan-out bursts.
- Bulk sends are expanded into individual per-user messages on the queue, mirroring Chapter 8's fan-out-on-write pattern.
- Retry with exponential backoff and a bounded dead-letter path handles the reality of frequent transient third-party delivery failures.
- Deduplication operates at two distinct levels: caller-retry idempotency keys, and a separate cross-service near-duplicate dedup cache.
- Channel choice is a genuine cost trade-off — push-first with SMS as a fallback reflects real per-send cost differences.
- User preferences and opt-outs must be checked before queuing, not after, to avoid wasted cost and poor user experience.